

AMLGuard

Manual de Estudo do Projeto — do notebook ao pipeline MLOps

Detecção de lavagem de dinheiro sob desbalanceamento extremo
Dataset IBM AML HI-Small — 5.078.345 transações — 0,1019% positivos

Caio Bernardinelli — Técnico em IA (IFNMG) — Julho/2026
Documento de consulta pós-projeto para reforço do aprendizado

Como usar este manual (leia primeiro)

Este manual não é só um registro do que foi feito — é uma ferramenta de **revisão espaçada**. A memória de longo prazo se consolida quando você tenta **recuperar** a informação antes de relê-la, não quando apenas relê. Por isso, cada passo termina com um bloco de **autoteste**: pare, responda em voz alta ou por escrito, e só depois confira o texto. Errar no autoteste é parte do método — o erro sinaliza exatamente o que revisar.

Sugestão de ciclo de revisão:

- **1 dia** após a leitura: refaça apenas os autotestes, sem ler o texto.
- **1 semana** depois: releia só os blocos ANALOGIA e LIÇÃO; refaça os autotestes que errou.
- **1 mês** depois (e antes de entrevistas): releia a Parte 4 (modelos) inteira — é o material com maior chance de cair em entrevista técnica.
- Sempre que voltar ao repositório após semanas parado: releia a Parte 3 (Fase MLOps).

Sumário

Parte 1 — O problema e o contexto de negócio

Parte 2 — Cronologia da Fase 1: o notebook acadêmico (passos 1 a 10)

Parte 3 — Cronologia da Fase 2: refatoração MLOps (Dias 1 a 6)

Parte 4 — Os modelos preditivos: por quê, alternativas, quando falham, como confirmar

Parte 5 — Os dois bugs reais que os portões pegaram (estude-os!)

Parte 6 — Glossário rápido de consulta

Parte 1 — O problema e o contexto de negócio

Bancos são obrigados por lei a detectar transações ligadas à lavagem de dinheiro. Na União Europeia, a pressão aumentou com a AMLA (Anti-Money Laundering Authority), autoridade central em Frankfurt que assumiu os mandatos de combate à lavagem em janeiro de 2026. O problema prático dos bancos não é falta de alertas — é **excesso de alertas falsos**: os sistemas tradicionais (baseados em regras) sinalizam tanta coisa que as equipes de compliance não conseguem revisar tudo.

O núcleo técnico do problema: menos de 1% das transações são ilícitas. No nosso dataset, **0,1019%** — cerca de 1 em cada 981. Isso se chama **desbalanceamento extremo de classes**, e quebra a intuição usual sobre o que é um modelo "bom".

ANALOGIA DO MUNDO REAL — o vigia que nunca erra

Imagine um vigia de estacionamento que, para "acertar mais", decide dizer todos os dias: *"hoje ninguém vai roubar carro nenhum"*. Como furtos são raros, ele acerta 99,9% dos dias — e é completamente inútil, porque nunca pega um ladrão. É exatamente isso que um modelo avaliado só por **acurácia** faz num problema desbalanceado. O AMLGuard existe para construir o vigia que aponta os carros certos a vigiar, aceitando errar algumas vezes.

AUTOTESTE — responda antes de reler

1) Por que 99,9% de acurácia pode ser um resultado péssimo aqui? 2) Qual é a taxa de positivos do dataset e por que ela determina a escolha das métricas?

Parte 2 — Cronologia da Fase 1: o notebook acadêmico

Passo 1 — Escolha do dataset e setup de reprodutibilidade

Escolhemos o **IBM Transactions for Anti-Money Laundering (HI-Small)**: 5.078.345 transações sintéticas geradas por um simulador multiagente, com rótulo de lavagem por transação e licença aberta. "Sintético" significa gerado por um programa, não coletado de um banco real — ótimo para aprender e publicar (sem dados sensíveis), mas exige cautela: sinais fortes demais podem refletir regras do gerador, não comportamento real de lavadores.

Antes de qualquer modelo, fixamos **RANDOM_STATE = 42** como semente global e configuramos o `.gitignore` para nunca commitar o CSV de 476 MB.

ANALOGIA DO MUNDO REAL — dados sintéticos são um simulador de voo

Treinar com dados sintéticos é como treinar pilotos num simulador de voo: a física do simulador é parecida com a real, mas não idêntica. Um piloto excelente no simulador provavelmente é bom no ar — mas não há garantia sem voar de verdade. Por isso o notebook repete em toda conclusão forte: *"isso pode ser um artefato do gerador"*.

AUTOTESTE — responda antes de reler

Por que fixar a semente aleatória ANTES de escrever qualquer modelo? O que quebraria sem ela?

Passo 2 — Carga, análise descritiva e tratamento

Carga com pandas, verificação de tipos, memória e da coluna-alvo **Is Laundering** (0 = legítima, 1 = lavagem). A análise descritiva quantificou o achado central: **99,8981% vs 0,1019%**. O tratamento converteu timestamps, checkou duplicatas e ausentes, e documentou cada decisão no formato problema → decisão → justificativa.

ANALOGIA DO MUNDO REAL — inventário antes da reforma

É como receber uma casa antes de reformar: antes de derrubar qualquer parede, você percorre todos os cômodos anotando o que existe, o que está quebrado e o que é estrutural. Modelar sem essa fotografia é reformar no escuro.

AUTOTESTE — responda antes de reler

Qual a diferença entre análise descritiva (este passo) e análise exploratória (o próximo)? Por que separá-las?

Passo 3 — EDA e engenharia de features — a pergunta-guia

Toda a exploração girou em torno de uma pergunta: **o que distingue o 0,1% ilícito?** A régua de comparação foi sempre a **taxa ilícita dentro do grupo** versus a taxa global (0,1019%) — nunca contagens brutas, que enganam em grupos grandes.

Features **selecionadas**: **Payment Format** (ACH concentrava 86,6% dos ilícitos, com taxa 7,3x a base), **Amount Paid** (baseline monetário), **sender_previous_tx_count** (velocidade histórica da

conta), **is_business_hours** (contraintuitivo: ilícitos aparecem MAIS em horário comercial, lift 1,38x) e **same_account** (mantida provisoriamente; fortemente confundida com o formato Reinvestment).

Features **excluídas** com justificativa: **is_cross_currency** (0 ilícitos em 72 mil casos — cheiro de artefato do gerador), **round_value** (627 casos em 5 milhões — sem sinal), **same_bank** (correlação 0,914 com **same_account** — redundante), **Amount Received** (correlação 0,843 com **Amount Paid**) e os IDs de conta/banco (risco de memorização de entidades específicas).

ANALOGIA DO MUNDO REAL — taxa dentro do grupo, não contagem

Se 90% dos acidentes de carro acontecem perto de casa, isso NÃO significa que perto de casa é perigoso — é onde as pessoas mais dirigem. O que importa é a taxa de acidentes POR km rodado em cada lugar. Pelo mesmo motivo, comparamos a taxa ilícita dentro de cada grupo com a taxa global, nunca a contagem bruta.

LIÇÃO / ERRO REAL DO PROJETO — correção de leakage na feature de velocidade

A primeira versão contava as transações de cada conta usando o dataset INTEIRO — incluindo transações futuras e do conjunto de teste. Isso é **vazamento de dados (leakage)**: informação que o modelo não teria na vida real. Foi substituída por uma versão histórica: ordenar por tempo e contar apenas as transações ANTERIORES da conta (cumcount). Grave esta distinção — ela reaparece na Fase 2.

AUTOTESTE — responda antes de reler

Explique com suas palavras por que contar transações no dataset inteiro é leakage, mas contar só as anteriores não é. O que o modelo "saberia do futuro" na versão errada?

Passo 4 — Preparação dos dados: split e pipelines

Split **estratificado** 70/30 (3.554.841 treino / 1.523.504 teste), garantindo a mesma proporção de ilícitos dos dois lados — sem stratify, o teste poderia ter tão poucos positivos que as métricas virariam ruído. Todo preprocessing (OneHotEncoder para categóricas, StandardScaler para numéricas no modelo linear) vive DENTRO de pipelines scikit-learn e é ajustado APENAS no treino.

ANALOGIA DO MUNDO REAL — a prova que o aluno nunca viu

Treino/teste é professor e aluno: você ensina com uma lista de exercícios (treino) e avalia com uma prova inédita (teste). Ajustar o preprocessing no dataset inteiro é deixar o aluno espiar a prova antes: a nota sobe, mas não mede aprendizado. Estratificar é garantir que a prova tenha a mesma proporção de questões difíceis que a lista de exercícios.

AUTOTESTE — responda antes de reler

O que quebraria se o StandardScaler fosse ajustado no dataset inteiro antes do split?

Passo 5 — Baselines: Dummy e Regressão Logística (balanceada e sem pesos)

Dummy Classifier (prever tudo como legítimo): 99,8981% de acurácia, ZERO lavagens detectadas, Average Precision 0,001019 (igual à prevalência — nenhuma capacidade de ranking). Existe para provar, no próprio notebook, que acurácia isolada engana.

Logistic Regression balanceada (`class_weight="balanced"`): recall 90,28% (1.402 de 1.553 lavagens), mas 192.418 alertas (12,63% de tudo!) com precisão de 0,73% — 1 lavagem real a cada 137 alertas revisados. AP = 0,008381 (8,2x o Dummy): há sinal real nas features.

Logistic Regression SEM pesos (experimento de controle): no threshold 0,50 colapsa para o comportamento do Dummy (0 alertas), MAS a AP foi 0,009182 — **a maior das três**. Descoberta metodológica central do projeto: **class_weight não melhorou o ranking**; só deslocou onde o corte 0,50 cai na distribuição de scores. Reponderar classes é, na prática, uma intervenção de threshold disfarçada de intervenção de modelo.

AUTOTESTE — responda antes de reler

A LogReg sem pesos gerou 0 alertas e mesmo assim teve AP maior que a balanceada. Como isso é possível? (Dica: AP mede ranking; threshold mede corte.)

Passo 6 — Modelos de árvore: Random Forest e XGBoost

A justificativa para sair do modelo linear foi específica, não "tentar outra coisa": os sinais da EDA eram **não lineares e de interação** (ACH com lift 7,3x, velocidade não monotônica, same_account confundida com Reinvestment). Modelos lineares não capturam limiares nem interações; árvores sim.

Random Forest (`balanced_subsample`): AP 0,0325, recall 81,8%, taxa de alertas 5,67%. **XGBoost** (`scale_pos_weight = neg/pos ≈ 980`): **AP 0,036833 — o melhor ranking de todos**. **XGBoost com undersampling 1:10**: AP 0,0333 — mais rápido de treinar, ranking levemente pior.

A comparação final entre modelos foi feita por **Average Precision** (independente de threshold), nunca pelas métricas no corte 0,50.

Modelo	AP	Recall @0,50	Precisão @0,50	Alertas
XGBoost — Scale Pos Weight	0,0368	89,6%	0,82%	11,2%
XGBoost — Undersampling 1:10	0,0333	60,3%	2,19%	2,81%
Random Forest — Bal. Subsample	0,0325	81,8%	1,47%	5,67%
LogReg — Sem pesos	0,0092	0,0%	—	0,0%
LogReg — Balanceada	0,0084	90,3%	0,73%	12,6%
Dummy (mais frequente)	0,0010	0,0%	—	0,0%

AUTOTESTE — responda antes de reler

Por que comparar modelos pela AP e não pelo recall no threshold 0,50? O que a tabela esconderia se olhássemos só recall?

Passo 7 — Avaliação operacional: threshold como decisão de negócio

Escolhido o XGBoost, o threshold foi tratado como uma decisão de **orçamento de analistas**, não como número mágico. A pergunta de negócio: *"quanta lavagem conseguimos pegar se a equipe*

consegue revisar 0,5%, 1%, 2% ou 5% das transações?"

Ponto de operação escolhido: **precisão mínima de 2%** → threshold 0,892163 → **recall 67,0%** (1.041 de 1.553 lavagens), 52.044 alertas (3,42% do teste), ~50 alertas revisados por lavagem encontrada. Os 512 falsos negativos são o risco regulatório residual — declarado, não escondido.

ANALOGIA DO MUNDO REAL — o botão de sensibilidade do detector de fumaça

O threshold é o botão de sensibilidade de um detector de fumaça. Sensibilidade máxima: pega todo incêndio, mas dispara com torrada queimada dez vezes por dia (e a família passa a ignorá-lo — o que é pior que não ter alarme). Sensibilidade mínima: silêncio ótimo, até a casa pegar fogo. Não existe posição "correta" do botão: existe a posição certa PARA a sua tolerância a incêndio e a sua paciência com alarmes falsos. Em compliance: tolerância à lavagem não detectada versus capacidade de revisão da equipe.

AUTOTESTE — responda antes de reler

Se a equipe de compliance dobrar de tamanho, o threshold deve subir ou descer? O que acontece com recall e precisão?

Passo 8 — Explicabilidade com SHAP

SHAP foi executado somente no modelo final, sobre uma amostra (nunca nos 5 milhões de linhas). Saídas: summary plot global (Payment Format ACH e sender_previous_tx_count dominam) e explicações locais do alerta verdadeiro-positivo e do falso-positivo de maior risco. Ressalva registrada: explicação não é causalidade, e features fortes podem codificar regras do gerador sintético.

ANALOGIA DO MUNDO REAL — a conta itemizada do restaurante

Um score de risco sem SHAP é uma conta de restaurante que só diz o total: R\$ 347. O SHAP é a conta itemizada: entrada R\$ 40, vinho R\$ 180, sobremesa R\$ 45... Você pode discordar do preço do vinho, mas agora dá para AUDITAR. Para um analista de compliance decidir se investiga, saber O QUE puxou o score para cima vale tanto quanto o próprio score.

AUTOTESTE — responda antes de reler

Por que rodar SHAP sobre uma amostra, e não sobre o dataset inteiro? E por que só no modelo final?

Passo 9 — Conclusão crítica e entrega acadêmica

O notebook fechou com as 9 seções da rubrica e uma conclusão crítica honesta: dataset sintético, janela de 18 dias, split aleatório que não simula deployment futuro (as mesmas contas podem aparecer em treino e teste), necessidade futura de validação temporal e de features de grafo. Entrega em 05/07/2026, dentro do prazo.

AUTOTESTE — responda antes de reler

Cite de memória 3 limitações declaradas na conclusão. Por que declará-las FORTALECE o projeto num portfólio?

Passo 10 — Publicação para recrutadores

README completo com contexto AMLA, tabela de resultados, 4 gráficos extraídos do notebook, metodologia, limitações e roadmap; LICENSE MIT; badges; keywords ATS (fraud detection, imbalanced classification, XGBoost, SHAP, precision-recall...); topics no GitHub; elegibilidade para trabalhar na UE declarada.

AUTOTESTE — responda antes de reler

Qual é a primeira coisa que um recrutador vê ao abrir o repositório? (Resposta: o README renderizado — por isso ele importa mais que qualquer célula do notebook.)

Parte 3 — Cronologia da Fase 2: refatoração MLOps (Dias 1 a 6)

A entrega acadêmica provou O QUE o modelo faz. A Fase 2 transforma o notebook em software: scripts, testes, contratos e portões de regressão. O princípio central: **congelar o baseline validado e exigir que toda refatoração o reproduza**.

Passo D1 — Estrutura profissional + baseline congelado

Criada a estrutura de pacote (src/data, src/features, src/models, src/api, tests, artifacts, docs), o config.py central (caminhos relativos, semente, listas de features, contrato do modelo) e o arquivo mais importante da fase: **artifacts/baseline_metrics.json** — todas as métricas validadas do notebook, congeladas, com um portão de aceitação ($AP \geq 0,035$; $recall \geq 0,65$).

ANALOGIA DO MUNDO REAL — o quilograma-padrão

Durante 130 anos, "um quilo" foi definido por UM cilindro de platina guardado num cofre em Paris. Toda balança do mundo era, em última instância, comparada a ele. O baseline_metrics.json é o nosso cilindro de platina: qualquer versão futura do código é pesada contra ele. Se o resultado divergir, o problema é do código novo — o padrão não se move.

AUTOTESTE — responda antes de reler

Por que congelar o baseline ANTES de refatorar, e não depois? O que se perde fazendo ao contrário?

Passo D2 — load_data.py — validação de schema

Módulo de carga que valida o CSV ANTES de qualquer processamento: colunas obrigatórias presentes, alvo estritamente binário, colunas de valor numéricas. Falha com mensagem que NOMEIA a coluna problemática (SchemaValidationError), nunca com um erro cru de pandas no meio do pipeline.

ANALOGIA DO MUNDO REAL — o segurança na porta da festa

Validar schema na entrada é o segurança conferindo identidade na porta: barrar um penetra na entrada custa 10 segundos; descobrir o penetra no meio da festa custa a festa. Um CSV com coluna faltando que "entra" silenciosamente só explode 20 minutos depois, num erro incompreensível dentro do XGBoost.

AUTOTESTE — responda antes de reler

Cite os 3 tipos de violação de schema que o load_data detecta. Por que importa tanto a mensagem de erro nomear a coluna?

Passo D3 — Paridade de features — e o primeiro bug pego

Script de verificação com dupla prova: (1) PARIDADE — cada feature do pacote é recomputada por uma implementação independente copiada do notebook e comparada byte a byte; (2) NO-TARGET — análise da árvore sintática (AST) provando que nenhum código lê a coluna-alvo, mais a prova estrutural de que construir features sem a coluna-alvo funciona.

O check de paridade **REPROVOU na primeira execução** — e pegou um bug real (ver Parte 5, Bug 1).

AUTOTESTE — responda antes de ler

O que é melhor: um teste que sempre passa ou um teste que reprova um bug real na primeira rodada? Por quê?

Passo D4 — train.py reproduzível — e o segundo bug pego

Script de treino executável (python -m src.models.train) com hiperparâmetros congelados no config, scale_pos_weight derivado do treino, checkpoint do modelo salvo ANTES da avaliação e portão contra o baseline. A prova de determinismo treinou duas vezes e comparou o SHA-256 dos artefatos: idênticos.

Resultado real na sua máquina: AP 0,036098 (baseline 0,036833), recall 66,1% — **PASS**. O microdesvio (~1%) veio de versões de biblioteca; o portão foi projetado com tolerância para isso.

No caminho, o portão pegou o segundo bug real do projeto (ver Parte 5, Bug 2).

ANALOGIA DO MUNDO REAL — a receita de bolo determinista

Determinismo é assar o mesmo bolo duas vezes com a mesma receita, os mesmos ingredientes e o mesmo forno — e obter bolos IDÊNTICOS. Se saírem diferentes, existe um ingrediente não anotado na receita (uma fonte de aleatoriedade não controlada). O hash SHA-256 é a "foto de alta resolução" que compara os dois bolos molécula a molécula.

AUTOTESTE — responda antes de ler

O que significa dois treinos produzirem o mesmo SHA-256? Que fontes de aleatoriedade tiveram de ser controladas?

Passo D5 — evaluate.py — dois portões independentes

Avaliação que recarrega o modelo salvo, reconstrói EXATAMENTE o mesmo conjunto de teste do treino (helper compartilhado prepare_train_test_split) e compara métrica a métrica contra o baseline congelado. Dois portões: **qualidade absoluta** (AP $\geq 0,035$ e recall $\geq 0,65$) e **regressão relativa** (cada métrica dentro de tolerância de 5–10% vs baseline). Exit code 0 só se ambos passam — pronto para CI/CD.

ANALOGIA DO MUNDO REAL — exame de saúde com histórico

O portão de qualidade é o exame com valores de referência da população ("colesterol abaixo de 200"). O portão de regressão é a comparação com o SEU exame anterior ("subiu 40% desde o ano passado"). Um valor pode estar dentro da faixa populacional e ainda assim sinalizar problema se mudou rápido demais. Os dois juntos pegam o que cada um sozinho deixaria passar.

AUTOTESTE — responda antes de ler

Dê um exemplo de situação que passaria no portão absoluto mas falharia no de regressão — e o contrário.

Passo D6 — predict.py — o contrato de predição

API de scoring unificada: predict_transaction e predict_batch, retorno padronizado {risk_score, is_alert, threshold, model_version}, modelo carregado UMA vez por processo (cache) e erro específico para input malformato. Decisão de design importante: **o predict NÃO faz feature engineering** — recebe features prontas, porque em produção sender_previous_tx_count viria de um feature store, e não seria calculado na hora do request.

ANALOGIA DO MUNDO REAL — o caixa e a cozinha

Num restaurante, o caixa (predict) não cozinha: recebe o prato pronto da cozinha (feature store) e entrega ao cliente com o ticket padronizado. Se o caixa tivesse que cozinhar cada pedido, o atendimento seria lento e cada caixa cozinaria diferente. Separar "preparar features" de "servir predições" é a mesma divisão de trabalho.

AUTOTESTE — responda antes de reler

Por que o cache do modelo importa numa API? O que aconteceria se o joblib fosse recarregado a cada request?

Parte 4 — Os modelos preditivos: as 4 perguntas

Para cada modelo usado no projeto: por que foi escolhido, que alternativas existiam, em que cenários a escolha seria ruim e que experimento confirmaria (ou refutaria) a escolha. Este é o material com maior chance de aparecer em entrevista técnica.

4.1 Dummy Classifier (estratégia: classe mais frequente)

Por que este modelo foi escolhido?

Não foi escolhido para prever — foi escolhido para **provar um ponto**: estabelecer o piso que todo modelo precisa superar e demonstrar, com números do próprio projeto, que acurácia é uma métrica enganosa sob desbalanceamento (99,9% de acurácia, zero lavagens detectadas).

Quais alternativas existiam?

- Não usar baseline nenhum (ruim: sem referência, qualquer AP parece boa ou ruim sem contexto).
- Baseline aleatório uniforme (menos didático: não maximiza a acurácia, então não expõe a armadilha).
- Regras de negócio simples (ex.: alertar todo ACH acima de X) — seria um baseline mais forte e mais próximo do que bancos realmente usam; ficou como reflexão para versões futuras.

Em quais cenários essa escolha seria ruim?

- Nunca é "ruim" incluir — o único erro possível seria PARAR nele ou reportá-lo como se fosse um modelo de verdade.

Que experimento confirmaria que foi a melhor opção?

O próprio projeto é o experimento: Dummy AP = 0,001019 (exatamente a prevalência) vs XGBoost AP = 0,0368 (36x). A distância entre os dois é a evidência de que os modelos aprenderam algo real.

ANALOGIA DO MUNDO REAL — o marco zero

O Dummy é o marco zero de uma corrida: ninguém o celebra, mas sem ele não dá para saber quanto qualquer corredor andou.

4.2 Regressão Logística (balanceada e sem pesos)

Por que este modelo foi escolhido?

Escolhida como primeiro modelo de verdade por três motivos: é **interpretável** (coeficientes legíveis), é **rápida** em 3,5 milhões de linhas e estabelece quanto do sinal é capturável por um modelo **linear** — o que dá significado ao ganho dos modelos de árvore depois. A dupla (com e sem `class_weight`) foi um experimento de controle deliberado para isolar o efeito da reponderação de classes.

Quais alternativas existiam?

- Naive Bayes (rápido, mas a suposição de independência entre features é claramente falsa aqui: Payment Format e same_account são correlacionados).
- SVM linear (custo computacional proibitivo em 3,5M de linhas; probabilidades mal calibradas).
- kNN (inviável: buscar vizinhos em milhões de pontos a cada predição; sofre com categóricas).
- Ir direto para o XGBoost (perderia o experimento de controle — não saberíamos se o ganho vem das features, do tratamento de desbalanceamento ou da não linearidade).

Em quais cenários essa escolha seria ruim?

- Quando as relações dominantes são não lineares ou de interação — exatamente o nosso caso, e por isso ela foi degrau, não destino.
- Quando as features têm escalas muito diferentes sem scaling (ela é sensível; árvores não).
- Quando se precisa de probabilidades calibradas direto da caixa com `class_weight` ligado — a reponderação distorce a calibração.

Que experimento confirmaria que foi a melhor opção?

O experimento já foi feito e é um dos achados centrais: comparar a AP da versão balanceada (0,0084) com a da sem pesos (0,0092) provou que `class_weight` NÃO melhora ranking — só move o threshold implícito. Para confirmar o teto linear: comparar a melhor LogReg (AP $\approx$ 0,009) com o XGBoost (AP 0,0368) — o salto de 4x é a evidência de que havia estrutura não linear fora do alcance do modelo linear.

ANALOGIA DO MUNDO REAL — a régua e a curva

A regressão logística desenha uma única linha reta separando "suspeito" de "normal". Se a fronteira real tem curvas e esquinas (ex.: "ACH E valor alto E conta nova" = suspeito, mas cada condição sozinha = normal), a régua não alcança. Ela mede o quanto da fronteira é reta — informação valiosa por si só.

4.3 Random Forest (balanced_subsample)

Por que este modelo foi escolhido?

Primeiro modelo não linear, escolhido porque a EDA encontrou sinais de limiar e interação que árvores capturam naturalmente. Random Forest é robusto, exige pouco tuning e dispensa scaling — um "primeiro não linear" de baixo risco. `balanced_subsample` repondera a classe rara POR ÁRVORE, adequado ao desbalanceamento.

Quais alternativas existem?

- Gradient boosting direto (foi o passo seguinte; começar pela floresta deu um degrau de comparação).
- Uma única árvore de decisão (interpretável, mas instável e fraca — alta variância).
- ExtraTrees (similar, com splits mais aleatórios; não mudaria a conclusão).
- Redes neurais (custo de tuning alto e pouca vantagem em dados tabulares — literatura e prática mostram árvores boostadas dominando o tabular).

Em quais cenários essa escolha seria ruim?

- Datasets pequenos, onde há risco de overfitting mesmo com bagging.
- Quando memória importa: florestas com centenas de árvores profundas em milhões de linhas consomem muito (o fit levou ~167 s vs ~47 s do XGBoost aqui).
- Quando se precisa de scores bem calibrados sem pós-processamento.
- Produção com latência crítica: centenas de árvores profundas custam mais por predição que as árvores rasas do boosting.

Que experimento confirmaria que foi a melhor opção?

Comparar AP em condições iguais — feito: RF 0,0325 vs XGBoost 0,0368. Para confirmar com rigor estatístico: validação cruzada estratificada (5 folds) comparando as APs médias com desvio — se os intervalos não se sobrepõem, a superioridade do XGBoost não é sorte do split.

ANALOGIA DO MUNDO REAL — o comitê de médicos independentes

Random Forest é um comitê de 100 médicos que estudaram em faculdades diferentes (cada árvore vê um subconjunto aleatório dos dados e das features) votando de forma independente. O comitê erra menos que qualquer médico sozinho — mas ninguém no comitê aprende com os erros dos colegas. Essa é a deixa para o próximo modelo.

4.4 XGBoost com `scale_pos_weight` — O MODELO FINAL

Por que este modelo foi escolhido?

Venceu pela métrica principal: **AP 0,036833**, o melhor ranking entre todos. Gradient boosting constrói árvores EM SEQUÊNCIA, cada uma corrigindo os erros residuais das anteriores — em vez do voto independente da floresta. `scale_pos_weight` ($\approx 980 = \text{neg/pos}$) faz cada exemplo raro de lavagem "pesar" como ~ 980 legítimos na função de perda, forçando o modelo a não ignorar a classe rara. `eval_metric="aucpr"` alinha o treino com a métrica que de fato importa (área precision-recall). `tree_method="hist"` torna o treino rápido E determinístico. Foi comparado também contra a própria variante com undersampling 1:10 (AP 0,0333) — a versão com `scale_pos_weight` manteve o melhor ranking.

Quais alternativas existiam?

- LightGBM (comparável ou mais rápido; a escolha do XGBoost foi por ubiquidade em fintech e nas vagas — keyword ATS relevante).
- CatBoost (trata categóricas nativamente, sem one-hot; forte candidato para uma versão futura).
- SMOTE/SMOTENC no lugar de `scale_pos_weight` (rejeitado como estratégia principal: em 3,5M de linhas o custo de memória é alto, sintetizar pontos após one-hot cria categorias fracionárias, e a literatura moderna mostra pouca vantagem sobre reponderação em árvores boostadas).
- Redes neurais tabulares/TabNet (custo-benefício ruim aqui).
- Modelos de grafo/GNN (promissores para lavagem — é um fenômeno de rede — mas escopo de outra fase; está no roadmap).

Em quais cenários essa escolha seria ruim?

- Quando a interpretabilidade regulatória exigir modelo intrinsecamente interpretável (alguns reguladores preferem scorecards lineares; SHAP mitiga, mas não elimina a discussão).
- Datasets minúsculos (boosting overfita rápido com poucos dados).
- Quando o drift é rápido e retreinar é caro — boosting é mais sensível a mudanças de distribuição do que regras simples.
- Se a latência de predição precisasse ser submilissegundo em hardware limitado.

Que experimento confirmaria que foi a melhor opção?

Três experimentos, em ordem de custo: (1) validação cruzada estratificada comparando a AP de XGBoost vs RF vs LightGBM vs CatBoost — confirma a escolha entre famílias; (2) busca de hiperparâmetros (`learning_rate`, `max_depth`, `n_estimators`) com validação separada — confirma que o ponto atual não é um mínimo local ruim; (3) o mais importante, já no roadmap: **validação temporal** (treinar nos primeiros ~ 14 dias, testar nos últimos ~ 4) — confirma que o ranking sobrevive ao cenário real de "prever o futuro", que o split aleatório não simula.

ANALOGIA DO MUNDO REAL — a equipe de consultores em série

Se a Random Forest é um comitê votando em paralelo, o XGBoost é uma equipe de consultores em SÉRIE: o primeiro entrega um diagnóstico; o segundo lê os erros do primeiro e corrige só o que ficou errado; o terceiro corrige o que sobrou; e assim 200 vezes. Cada consultor é simples (árvore rasa), mas a sequência inteira fica precisa. O `scale_pos_weight` é a instrução dada a todos: "errar uma lavagem custa 980x mais que errar uma transação normal — priorizem de acordo".

Parte 5 — Os dois bugs reais que os portões pegaram

Estes dois bugs merecem mais estudo que qualquer seção que "deu certo de primeira". Ambos são silenciosos (não geram erro de execução), ambos destroem o modelo, e ambos foram pegos por verificações construídas ANTES de serem necessárias.

Bug 1 — Drift de dtype na migração das features (pego no Dia 3)

Ao migrar o código de features do notebook para o pacote, foram adicionadas duas "otimizações" que o notebook nunca teve: converter Amount Paid para float32 e Payment Format para category. Parece inofensivo — mas float64→float32 muda a quinta casa decimal dos valores, o que muda os splits das árvores, o que muda o modelo. O check de paridade byte a byte reprovou na primeira execução.

LIÇÃO / ERRO REAL DO PROJETO — regra extraída

Refatorar = mover, NUNCA "melhorar de passagem". Toda otimização não validada é uma divergência do baseline esperando para acontecer. Se a otimização valer a pena, ela vira uma sessão própria, com seu próprio teste de regressão.

Bug 2 — Desalinhamento features × rótulos (pego no Dia 4)

O `build_model_features()` ordena as linhas por Timestamp (necessário para a contagem histórica ser leakage-safe). O `train.py` extraía os rótulos do DataFrame ORIGINAL, na ordem do CSV. Resultado: as features da transação A eram treinadas com o rótulo da transação B — rótulos essencialmente embaralhados. O modelo treinou "normalmente", sem nenhum erro... e a AP colapsou para 0,0011, exatamente a taxa-base. O portão contra o baseline congelado reprovou, e a investigação chegou ao bug em minutos (pista extra: o modelo salvo tinha 0,3 MB — pequeno demais para ter aprendido algo).

ANALOGIA DO MUNDO REAL — provas embaralhadas

É como um professor corrigindo provas que caíram no chão e foram remontadas fora de ordem: cada prova recebe a nota de OUTRO aluno. Nenhuma regra de correção foi violada — o processo "roda sem erro" — mas o resultado não mede nada. Notas aleatórias, alunos revoltados, e nenhuma mensagem de erro em lugar nenhum.

LIÇÃO / ERRO REAL DO PROJETO — regra extraída

Sempre que features e rótulos são extraídos separadamente, a ordenação é um contrato entre eles. A correção foi criar UM helper (`prepare_train_test_split`) que ordena o DataFrame primeiro e extrai os dois da mesma fonte ordenada — e que `train.py` e `evaluate.py` são OBRIGADOS a compartilhar. Bugs silenciosos exigem portões numéricos: nenhum olho humano pega isso em code review.

AUTOTESTE — responda antes de reler

- 1) Por que nenhum dos dois bugs gera erro de execução? 2) Que sinal numérico denunciou cada um?
- 3) Qual mecanismo do projeto os pegou — e o que teria acontecido sem ele?

Parte 6 — Glossário rápido de consulta

Termo	Definição em uma linha
Average Precision (AP)	Área sob a curva precision-recall (average_precision_score). Mede qualidade de RANKING, independe de threshold. Métrica principal do projeto. Piso teórico = prevalência (0,001019).
Recall	Das lavagens reais, quantas o modelo pegou. Recall 67% = 1.041 de 1.553.
Precisão	Dos alertas emitidos, quantos eram lavagem real. Precisão 2% = 1 verdadeiro a cada 50 alertas.
Threshold	Nota de corte sobre o score. Acima = alerta. É decisão de NEGÓCIO (orçamento de analistas), não propriedade do modelo.
Leakage	Informação do futuro ou do teste vazando para o treino. Infla métricas offline; desaba em produção.
Split estratificado	Divisão treino/teste preservando a proporção das classes. Inegociável sob desbalanceamento.
class_weight / scale_pos_weight	Reponderação da classe rara na função de perda. Lição do projeto: move o threshold implícito, NÃO melhora o ranking.
Undersampling	Descartar exemplos da classe majoritária (1:10 testado). Treino mais rápido; ranking levemente pior aqui.
SMOTE	Sintetizar exemplos da classe rara. Rejeitado como estratégia principal: custo de memória em 5M de linhas e problemas com categóricas one-hot.
SHAP	Decomposição do score em contribuições por feature. A "conta itemizada" do modelo.
Baseline congelado	Métricas validadas gravadas em JSON como referência imutável. O "quilograma-padrão" do projeto.
Portão de regressão	Comparação automática métrica a métrica contra o baseline, com tolerância. Exit code $\neq 0$ bloqueia o deploy.
Determinismo	Mesmo código + mesmos dados + mesma semente = artefato byte-idêntico (mesmo SHA-256).
Feature store	Sistema que serve features pré-computadas em produção. Motivo pelo qual o predict.py não calcula features.
Schema validation	Conferir a estrutura dos dados NA ENTRADA, com erros que nomeiam a coluna. O "segurança da porta".

Fim do manual. Próxima revisão sugerida: 1 dia (autotestes), 1 semana (analogias + lições), 1 mês e pré-entrevista (Parte 4 completa).